

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Vietnam National University – Ho Chi Minh City · Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering



NỀN TẢNG LẬP TRÌNH CHO PHÂN TÍCH VÀ TRỰC QUAN DỮ LIỆU

Programming Foundations for Data Analysis and Visualization

ASSIGNMENT REPORT · PART 1 OF 3 · TABULAR DATA

Predicting Hotel Booking Cancellations

Which bookings will be cancelled — and how not to fool ourselves when measuring it

Lecturer **Dr. Lê Thành Sách**
Group **G3**
Semester 261, academic year 2026–2027

No.	Full name	Student ID	Role
1	Ly Minh Trung	2570349	Data Scientist
2	Dinh Truong Tue Linh	2570441	Data Scientist
3	Nguyen Hoang Nam	2570261	Analysis and Visualization

[Project page](#) · [Notebook on GitHub](#) · Video: to be published

Ho Chi Minh City, October 2026



Contents

1	Problem and objective	2
2	Dataset	3
3	Exploratory data analysis	4
4	Data preparation	6
5	Modelling	7
6	Results on the test set	8
7	Extra experiments	9
8	Conclusions	12

1 Problem and objective

Hotels take bookings weeks or months in advance, yet 37.0% of the bookings in our dataset were cancelled. A hotel that can tell, at booking time, which reservations are likely to be cancelled can overbook in a controlled way, ask guests to reconfirm, or require a deposit for risky bookings.

Objective. Build a binary classifier that predicts `is_canceled` (1 = cancelled) using only information that is available when the booking is made, and measure honestly how well it works on bookings it has never seen.

How success is measured. Cancellations are the minority class, so accuracy is misleading: a model that always answers “not cancelled” is already 72.4% accurate on the cleaned data while catching no cancellation at all. We therefore report **F1, recall, ROC-AUC and PR-AUC** (average precision), select models on a validation set, and touch the test set only once.

2 Dataset

Hotel Booking Demand [1] contains real bookings from two hotels in Portugal — a *City Hotel* in Lisbon and a *Resort Hotel* in the Algarve — with arrival dates from July 2015 to August 2017 (licence CC BY 4.0). Each row is one booking: 119,390 rows and 32 columns, of which 20 are numerical and 12 categorical.

Table 1. Column groups of the dataset.

Group	Columns	Meaning
Target	is_canceled	1 if the booking was cancelled
Timing	lead_time, arrival_date_*	days booked in advance; arrival date
Stay	stays_in_weekend_nights, stays_in_week_nights, adults, children, babies	nights and guests
Channel & guest	market_segment, distribution_channel, customer_type, country, agent, company, is_repeated_guest	booking source, customer type, nationality
History	previous_cancellations, previous_bookings_not_canceled	the guest's past behaviour
Service & price	meal, reserved_room_type, deposit_type, adr, required_car_parking_spaces, total_of_special_requests, booking_changes, days_in_waiting_list	meal plan, room, deposit, average daily rate (ADR)
Leakage	reservation_status, reservation_status_date, assigned_room_type	known only <i>after</i> arrival or cancellation — dropped

Table 2. Assignment requirements for tabular data (section 4.2) and how this dataset meets them.

Requirement	This dataset	Status
At least 2,000 samples	119,390 bookings (85,982 after cleaning)	Met
At least 10 columns	32 columns	Met
Missing values	company 94.31%, agent 13.69%, country 0.41%, children; plus hidden Undefined values	Met
Categorical columns (encoding)	12, e.g. hotel, meal, country, market_segment, deposit_type	Met
Numerical columns (scaling)	20, e.g. lead_time, adr, stays_in_week_nights	Met
Harder data (preferred)	imbalance (63.0% / 37.0% raw, 72.4% / 27.6% after cleaning), outliers (ADR up to 5,400), 27.8% duplicates, target leakage	Met

3 Exploratory data analysis

3.1 Data quality

Table 3 lists every quality problem we found and what we did about it. Missing values are not always errors: a missing company means “not booked through a company”, so it becomes a binary flag rather than a deletion. The dataset also hides missing values behind the string `Undefined`.

Table 3. Data-quality issues and the action taken for each.

Issue	Affected	Action
Missing company	94.31% of rows	flag <code>has_company</code> (missing = no company)
Missing agent	13.69% of rows	flag <code>has_agent</code> (missing = booked directly)
Missing country, children	0.41% of rows; 4 rows	most-frequent / median imputation, fitted on train
Hidden Undefined	<code>meal</code> 0.98%; <code>market_segment</code> 2 rows; <code>distribution_channel</code> 5 rows	<code>meal</code> : Undefined = SC (per documentation); others → missing → most frequent
Duplicated rows	33,241 (27.8%)	removed before splitting
Invalid bookings	180 with 0 guests; 2 with $\text{ADR} < 0$ or $> 1,000$	removed
0-night bookings	715 rows	kept — day use is a valid booking
Leakage columns	3 columns	dropped (Section 7.1 shows why)

Removing duplicates changes the picture: the cancellation rate falls from 37.0% to 27.6%, because most copies were *cancelled* group bookings. In particular, 93% of the Non Refund bookings were copies (14,587 → 1,010). The clean data is therefore more imbalanced (72.4% / 27.6%).

3.2 Cancellation versus key features

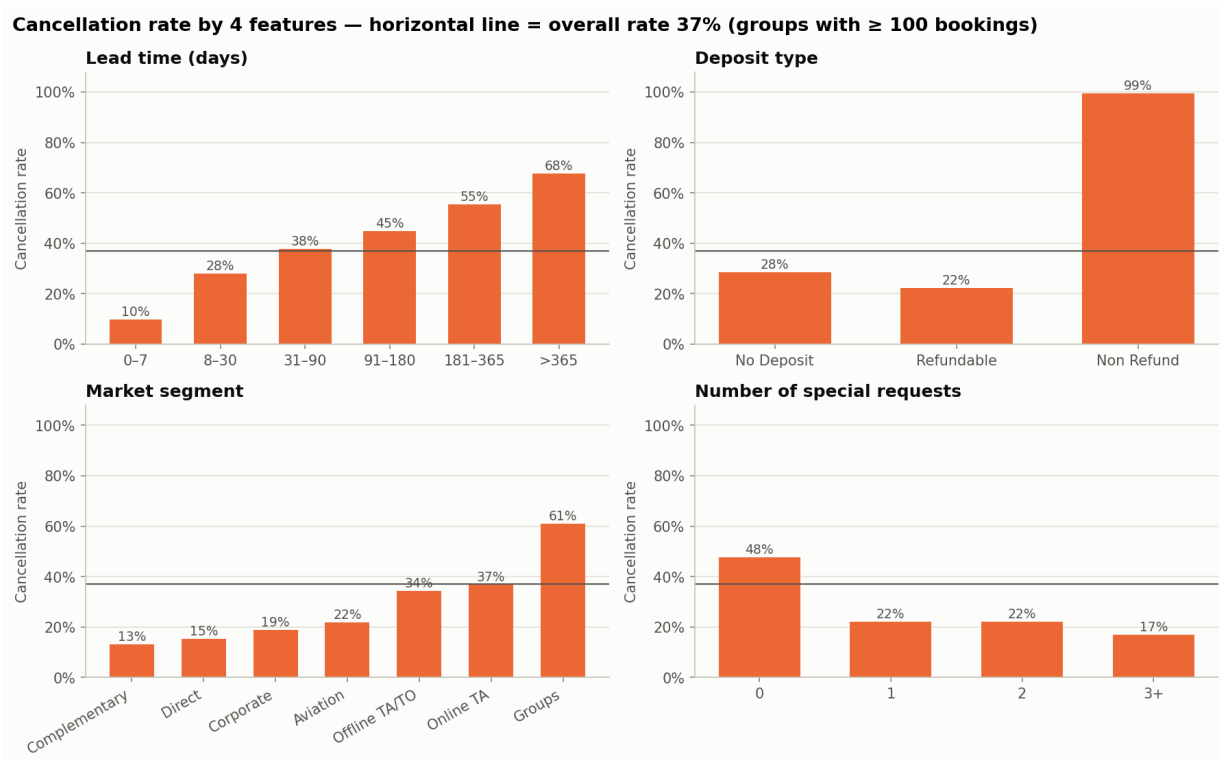


Figure 1. Cancellation rate by lead time, deposit type, market segment and number of special requests (raw data, groups with at least 100 bookings). The horizontal line is the overall rate.

Question. Which booking-time attributes are most strongly associated with cancellation?

Finding. **Lead time:** 9.6% of bookings made at most 7 days ahead are cancelled, versus 67.7% of bookings made more than a year ahead. **Non Refund** deposits are cancelled 99.4% of the time (vs 28.4% for no deposit) — counter-intuitive, and largely produced by the duplicated group blocks found above. **Groups** cancel most (61%), **Direct** (15%) and **Corporate** (19%) least. Guests with **special requests** cancel far less (47.7% with none vs 16.8% with three or more).

3.3 Distributions, outliers and correlation

The numerical columns are strongly right-skewed and contain thousands of points beyond the IQR fences. Most are real (bookings made two years ahead, expensive suites), so we do not delete them; we clip and transform them instead (Section 4). Spearman correlations with the target are only weak to moderate — `lead_time` +0.32, `previous_cancellations` +0.27, `total_of_special_requests` −0.26, `required_car_parking_spaces` −0.20 — so no single column decides the outcome and a multivariate, non-linear model is needed.

4 Data preparation

All steps that learn a parameter (medians, clipping quantiles, means and standard deviations, one-hot categories) are fitted on the training set only, inside a scikit-learn Pipeline with a ColumnTransformer. Row-level cleaning learns nothing from the data, so it is done before splitting. The notebook implements the steps as reusable classes: `HotelDataCleaner` (row cleaning with a log), `HotelFeatureBuilder` and `Winsorizer` (custom scikit-learn transformers) and `ModelBenchmark` (trains and scores many models the same way).

Table 4. Preparation pipeline.

Step	What is done	Fitted on
Row cleaning	drop leakage columns; remove invalid bookings; remove 33,226 duplicates	— (rules)
Feature engineering	<code>total_nights</code> , <code>total_guests</code> , <code>has_children</code> , <code>is_weekend_only</code> , <code>has_agent</code> , <code>has_company</code> , <code>is_domestic</code> , <code>arrival_weekday</code> ; cyclical <code>month_sin/month_cos</code>	— (stateless)
Missing values	median (numerical), most frequent (categorical)	train
Outliers	Winsorizer: clip to the 0.5%–99.5% quantiles	train
Skewness	<code>log1p</code> on count/day columns; not on <code>adr</code> (1.6% zeros)	—
Encoding	<code>OneHotEncoder</code> , categories below 0.5% merged into “infrequent” → 54 columns	train
Scaling	<code>StandardScaler</code> on the 26 numerical features	train
Split	stratified 70 / 15 / 15: 60,187 / 12,897 / 12,898 bookings	—

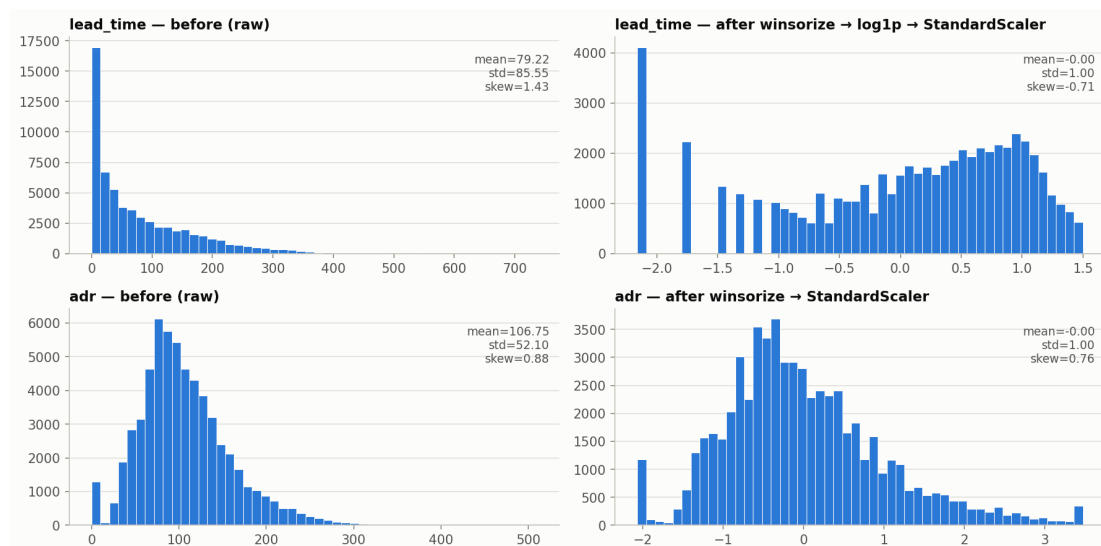


Figure 2. Distributions of `lead_time` and `adr` before and after preprocessing (training set).

Question. How does preprocessing change the two most important numerical features?

Finding. Both end with mean 0 and standard deviation 1. The skewness of `lead_time` drops from 1.43 to -0.71 and that of `adr` from 0.88 to 0.76. A first version also applied `log1p` to `adr` and made its skewness *worse* (-3.6) because complimentary rooms cost 0 — which is why `adr` has its own branch.

5 Modelling

5.1 Model comparison

Five classifiers share the same preprocessing pipeline: a dummy baseline, Logistic Regression, a Decision Tree (depth 12), a Random Forest (200 trees) and HistGradientBoosting (HGB).

Table 5. Validation scores (threshold 0.5). The tuned model is described in Section 5.2.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Dummy (always “not cancelled”)	0.724	0.000	0.000	0.000	0.500	0.276
Logistic Regression	0.795	0.675	0.493	0.570	0.847	0.684
Decision Tree	0.807	0.669	0.593	0.629	0.858	0.694
Random Forest	0.827	0.743	0.572	0.646	0.888	0.756
HistGradientBoosting (HGB)	0.826	0.723	0.596	0.653	0.887	0.753
HGB, tuned	0.828	0.721	0.613	0.662	0.894	0.768

The dummy model reaches 0.724 accuracy with $F1 = 0$, confirming that accuracy is not informative here. The two ensembles lead and are almost tied — HGB (F1 0.653, PR-AUC 0.753) and Random Forest (F1 0.646, PR-AUC 0.756) — while Logistic Regression is last (F1 0.570, PR-AUC 0.684) despite careful scaling: the signal is non-linear and involves interactions (a long lead time is risky only in some segments).

5.2 Hyperparameter tuning

HGB was tuned with `RandomizedSearchCV` (12 configurations $\times$ 3-fold stratified cross-validation on the training set, scored by PR-AUC). The best configuration — learning rate 0.05, 700 boosting iterations, 127 leaf nodes, at least 10 samples per leaf, L2 penalty 0.1 — reached a cross-validated PR-AUC of 0.766 and raised the validation PR-AUC from 0.753 to 0.768 (F1 from 0.653 to 0.662).

6 Results on the test set

The final model is the tuned HGB with the decision threshold chosen on the validation set (Section 7.2). It was evaluated once on the 12,898 test bookings.

Table 6. Test-set results of the baseline and the final model.

Model (test set)	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression (baseline)	0.800	0.689	0.501	0.580	0.853	0.694
HGB tuned, threshold 0.33	0.821	0.644	0.786	0.708	0.898	0.776

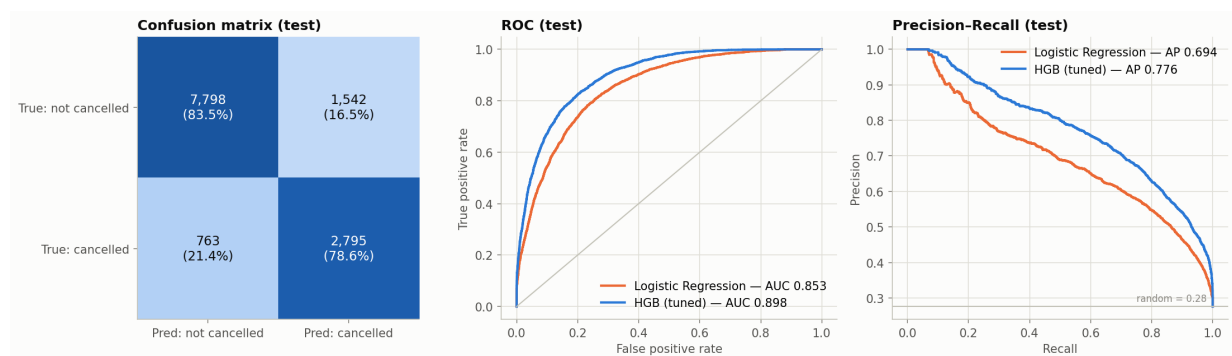


Figure 3. Final model on the test set: confusion matrix, ROC curve and precision–recall curve, compared with the Logistic Regression baseline.

Question. How good is the final model on bookings it has never seen?

Finding. F1 rises from 0.580 (baseline) to **0.708** (+0.13), ROC-AUC from 0.853 to **0.898** and PR-AUC from 0.694 to **0.776**. Of the 3,558 real cancellations the model catches 2,795 (recall 0.786); in exchange 1,542 bookings that were kept (16.5%) are flagged by mistake.

7 Extra experiments

7.1 Two evaluation traps: leakage and duplicates

Table 7. Validation scores of the same HGB model under three data-handling scenarios.

Scenario	Accuracy	F1	ROC-AUC	PR-AUC
Leakage columns kept	1.000	1.000	1.000	1.000
Duplicates not removed	0.854	0.793	0.933	0.904
Correct process (this report)	0.826	0.653	0.887	0.753

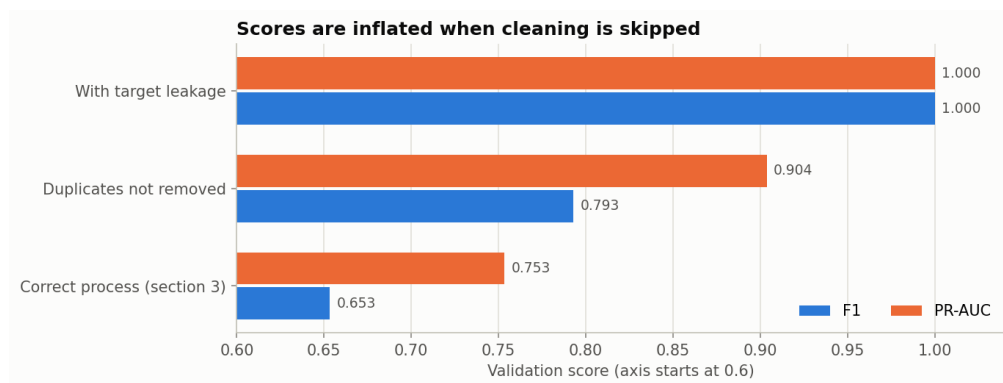


Figure 4. F1 and PR-AUC when the cleaning steps are skipped.

Question. If we skipped the cleaning of Section 4, how good would the scores look?

Finding. Keeping the leakage columns gives **1.000 on every metric**: the model just reads the answer from `reservation_status`. Keeping duplicates inflates F1 from 0.653 to 0.793 (+0.14) and PR-AUC from 0.753 to 0.904 (+0.15), because copies of the same booking appear in both training and validation. The honest figures are the lower ones.

7.2 Class imbalance and the decision threshold

Table 8. Three ways to handle the imbalance, on the tuned HGB model (validation set).

Strategy (validation set)	Precision	Recall	F1	PR-AUC
Default threshold 0.5	0.721	0.613	0.662	0.768
<code>class_weight=balanced</code>	0.613	0.814	0.700	0.768
Threshold tuned for F1 (0.33)	0.632	0.784	0.700	0.768

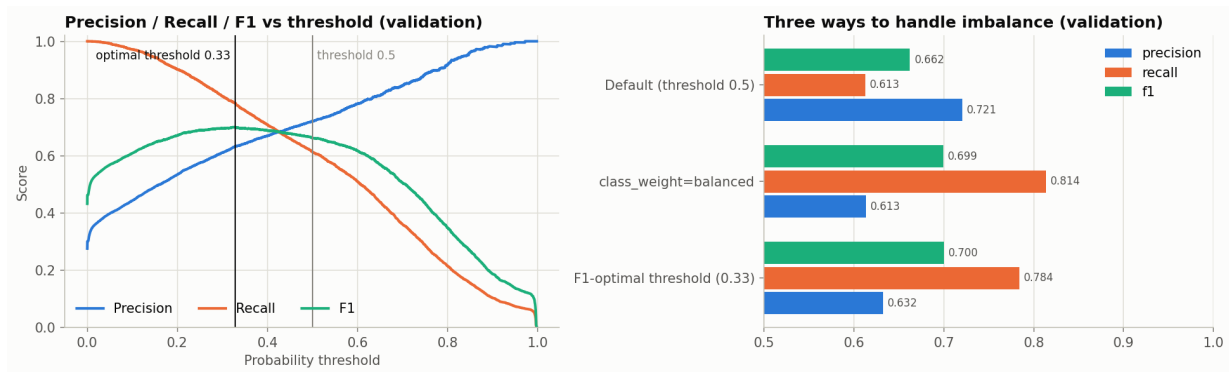


Figure 5. Left: precision, recall and F1 as the threshold moves. Right: the three strategies compared.

Question. Does handling the imbalance help, and how?

Finding. At threshold 0.5 the model misses many cancellations (recall 0.613). Re-weighting the classes raises recall to 0.814 and lowering the threshold to 0.33 raises it to 0.784; F1 rises from 0.662 to 0.700 with re-weighting and to 0.700 with the tuned threshold. PR-AUC barely moves (0.768): handling imbalance shifts the operating point along the same curve — trading precision for recall — rather than making the model better at ranking. We keep the tuned threshold because it needs no retraining and can later be set from the hotel’s real costs.

7.3 What the model relies on

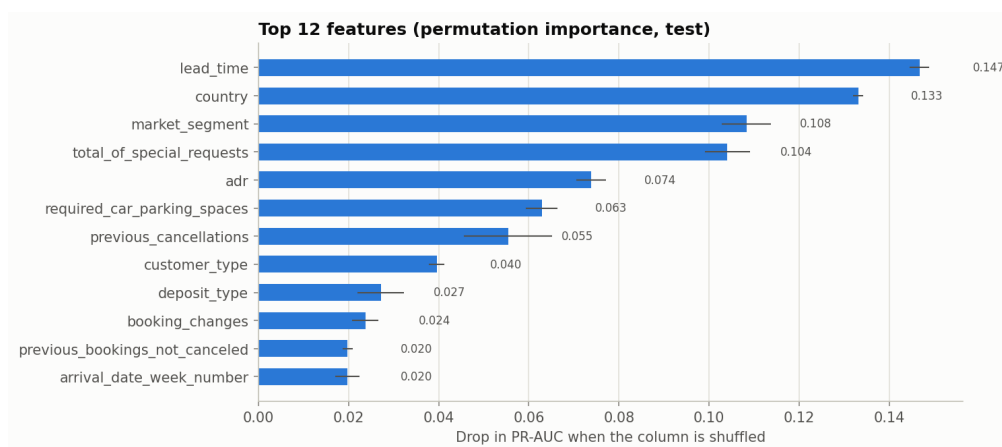


Figure 6. Permutation importance on the test set: drop in PR-AUC when one original column is shuffled.

Question. Which columns does the model depend on most?

Finding. lead_time (0.147) and country (0.133) matter most, followed by market_segment (0.108) and total_of_special_requests (0.104) — consistent with Figure 1. deposit_type ranks only 9th (0.027): once duplicates are removed, very few Non Refund bookings remain. Permutation importance shows what the model *uses*, not what *causes* a cancellation.

7.4 Where the model fails

Table 9. Test errors by market segment (segments with at least 50 test bookings).

Market segment	Test bookings	Cancel rate	Error rate	Recall
Online TA	7,693	35.8%	22.6%	0.81
Direct	1,794	13.0%	13.9%	0.41
Complementary	95	11.6%	13.7%	0.09
Corporate	571	12.6%	10.9%	0.42
Offline TA/TO	2,027	14.4%	8.9%	0.87
Groups	686	27.1%	8.5%	0.92

Question. Is the error rate the same for every kind of booking?

Finding. No. Errors concentrate in **Online TA** (error rate 22.6%), the largest segment and the one with the most cancellations (35.8%). For **Direct** and **Corporate** bookings recall is only 0.41 and 0.42: these guests rarely cancel, so the model seldom predicts a cancellation for them. Groups (0.92) and Offline TA/TO (0.87) are caught well. A per-segment threshold would be a natural next step.

8 Conclusions

1. **Cleaning decides whether results can be trusted.** 27.8% of the rows were duplicates and three columns leaked the answer; ignoring them gives “excellent” scores (F1 0.793–1.000) that are simply wrong. The true cancellation rate after cleaning is 27.6%, not 37.0%.
2. **Non-linear ensembles work best.** The tuned HistGradientBoosting model beats Logistic Regression by +0.13 F1 and +0.08 PR-AUC on the test set (F1 0.708, ROC-AUC 0.898).
3. **The threshold matters as much as the model.** Moving it from 0.5 to 0.33 raises validation recall from 0.613 to 0.784 at the same ranking quality.
4. **Strongest signals:** booking far ahead, nationality, sales channel, and *not* making special requests.

Limitations. The split is random rather than chronological, so the score on truly future bookings may be lower; the data covers two Portuguese hotels before COVID-19; and the F1-optimal threshold assumes that a missed cancellation and a false alarm cost the same.

Future work. A time-based split (train 2015–2016, test 2017); a threshold derived from real business costs; TargetEncoder for high-cardinality country and agent; probability calibration so scores can drive overbooking directly; per-booking explanations with SHAP.

References

- [1] N. Antonio, A. de Almeida, L. Nunes. *Hotel booking demand datasets*. Data in Brief, 22 (2019), 41–49. <https://doi.org/10.1016/j.dib.2018.11.126>
- [2] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12 (2011), 2825–2830.
- [3] Group G3. *Notebook G3_Tabular_HotelBooking.ipynb*. https://github.com/trungly-hcmut/C05177_DataVisualization_Assignment/blob/main/tabular/G3_Tabular_HotelBooking.ipynb
- [4] Group G3. *Project page (rendered notebook, report and video)*. https://trungly-hcmut.github.io/C05177_DataVisualization_Assignment/tabular/ — video: to be published